Retrieval-Augmented Coverless Semantic Steganography for Secure Semantic Communication

A report submitted for the course named Project - II (CS4202)

Submitted By

CHALUMURI NITHEESH PURNANAND
SEMESTER - VII
230103043

Supervised By

DR. JENNIL THIYAM



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF INFORMATION TECHNOLOGY SENAPATI, MANIPUR

NOVEMBER, 2026

Declaration

In this submission, I have expressed my idea in my own words, and I have adequately cited and referenced any ideas or words that were taken from another source. I also declare that I adhere to all principles of academic honesty and integrity and that I have not misrepresented or falsified any ideas, data, facts, or sources in this submission. If any violation of the above is made, I understand that the institute may take disciplinary action. Such a violation may also engender disciplinary action from the sources which were not properly cited or permission not taken when needed.

Chalumuri Nitheesh Purnanand
230103043

DATE: 05-11-2026



Department of Computer Science and Engineering
Indian Institute of Information Technology
Senapati, Manipur

Dr. Jennil Thiyam
Assistant Professor

Email: jennil@iiitmanipur.ac.in
Contact No: +91 8794521449

To Whom It May Concern

This is to certify that the project report entitled “Retrieval-Augmented Coverless Semantic Steganography for Secure Semantic Communication”, submitted to the department of Computer Science and Engineering, Indian Institute of Information Technology Senapati, Manipur in partial fulfillment for the award of degree of Bachelor of Technology in Computer Science and Engineering is record bonafide work carried out by Chalumuri Nitheesh Purnanand bearing roll number 230103043.

Signature of Supervisor

(Dr. Jennil Thiyam)

Signature of the Examiner 1 .....

Signature of the Examiner 2 .....

Signature of the Examiner 3 .....

Signature of the Examiner 4 .....

Abstract

Semantic steganographic communication hides a private image inside an innocent-looking image and sends it over a wireless channel, so that an eavesdropper does not even know a secret exists. The state-of-the-art scheme, AgentSemSteCom (IEEE TCCN, 2026), generates a reference image with a diffusion model for every message, and recovers the secret only approximately.

This project reproduces AgentSemSteCom end to end on a 6 GB laptop GPU and proposes **RACS** (Retrieval-Augmented Coverless Steganography). RACS obtains the reference image by deterministic retrieval from an indexed image corpus instead of generation, and adds an authenticated exact-recovery layer so the receiver gets the secret back pixel for pixel.

RACS obtains the reference about 82× faster (0.048 s instead of 3.92 s) with no GPU memory, keeps recovery quality and security (PSNR 22.62 vs 22.37 dB; token security gap 8.94 vs 8.67 dB), and reproduces the same reference at sender and receiver for all 66 test images. With the exact layer, all 66 secrets are recovered identically, every wrong key is rejected, and a classical steganalysis attack is reduced to chance. A phone prototype demonstrates the system on real photographs.

Acknowledgement

I would like to express my sincere gratitude to my supervisor, Dr. Jennil Thiyam, for the guidance, encouragement and constructive feedback provided throughout this project. Their insight into secure communication and generative models shaped both the direction and the rigour of the work.

I thank the Department of Computer Science and Engineering, Indian Institute of Information Technology Senapati, Manipur, for the opportunity and the academic environment that made this project possible. I am also grateful to the authors of AgentSemSteCom, EDICT, SwinJSCC, CodeFormer, CLIP and the other open-source projects used here, whose public code and checkpoints made a faithful reproduction feasible.

Finally, I thank my family and friends for their constant support and patience.

Chalumuri Nitheesh Purnanand

Contents

List of Figures **5**

List of Tables **6**

1 Introduction **7**
1.1 Outline 7
1.2 Problem Statement 8
1.3 Motivation 8
1.4 Purpose 8

2 Literature Survey **9**
2.1 Semantic Communication 9
2.2 Steganography and Steganalysis 9
2.2.1 Embedding-based steganography 9
2.2.2 Deep steganography 10
2.2.3 Coverless and generative steganography 10
2.2.4 CNN steganalysis 10
2.3 Diffusion Models and Exact Inversion 10
2.4 Conditioning, Perception and Restoration Models 11
2.5 Multimodal Retrieval 11
2.6 Evaluation Metrics 11
2.7 Research Gap 12

3 Requirement Engineering **13**
3.1 Stakeholders 13
3.2 Functional Requirements 13
3.3 Non-Functional Requirements 14
3.4 Hardware Requirements 15
3.5 Software Requirements 15
3.6 Models Used 15
3.7 Datasets 15
3.8 Constraints 16
3.9 Verification Gates 16

4 System Design **17**
4.1 Information Model . . . 17
4.2 Starting Point: the Baseline . . . 18
4.3 RACS Architecture . . . 18
4.4 New-Block Reference . . . 20
4.5 Prototype Architecture . . . 22
4.6 Code Organisation . . . 23

5 Implementation **24**
5.1 Methodology . . . 24
5.2 Foundation (Phases 1–3) . . . 25
5.2.1 Phase 1: environment and tensor contract . . . 25
5.2.2 Phase 2: VAE and latent space . . . 25
5.2.3 Phase 3: diffusion generation cost . . . 25
5.3 Retrieval Inputs and Test Harness (Phases 4–6) . . . 25
5.3.1 Phase 4: retrieval corpus . . . 25
5.3.2 Phase 5: CLIP embeddings . . . 25
5.3.3 Phase 6: evaluation harness and frozen splits . . . 26
5.4 Baseline Reproduction (Phases 7–16) . . . 26
5.4.1 Phase 7: public conditions . . . 26
5.4.2 Phase 8: reference generation . . . 26
5.4.3 Phase 9: ControlNet zero-convolution fine-tuning . . . 27
5.4.4 Phase 10: EDICT . . . 27
5.4.5 Phase 11: local hiding and recovery . . . 27
5.4.6 Phase 12: AWGN channel . . . 27
5.4.7 Phase 13: semantic codec . . . 28
5.4.8 Phase 14: CodeFormer enhancement . . . 28
5.4.9 Phase 15: assembled baseline . . . 28
5.4.10 Phase 16: security and steganalysis . . . 28
5.5 RACS Reference Retrieval (Phases 17–22) . . . 28
5.5.1 Phase 17: exact retrieval . . . 28
5.5.2 Phase 18: token selection . . . 29
5.5.3 Phase 19: structural representation . . . 29
5.5.4 Phase 20: reranking . . . 29
5.5.5 Phase 21: RACS end to end . . . 30
5.5.6 Phase 22: controlled comparison . . . 30
5.6 RACS Exact-Recovery Layer . . . 30
5.7 Prototype . . . 31
5.8 Verification Results . . . 32
5.9 Engineering Challenges . . . 33

6 Results **34**
6.1 Experimental Setup 34
6.2 Baseline Recovery Quality 34
6.3 Baseline Security and Steganalysis 35
6.4 RACS versus the Baseline 35
6.4.1 Protocol invariant 35
6.4.2 Efficiency 35
6.4.3 Quality and security 36
6.4.4 Hypotheses 37
6.5 RACS Exact-Recovery Layer 37
6.6 Prototype Performance 39
6.7 Discussion 39
6.7.1 Why retrieval does not cost quality 39
6.7.2 Security of the token 40
6.7.3 When RACS may not fit 40
6.8 Deviations from the Paper 41
6.9 Limitations 41

7 Conclusion **42**
7.1 Summary 42
7.2 Learning Outcomes 42
7.3 Future Work 43
7.4 Conclusion 44

Bibliography **45**

List of Figures

4.1 RACS sender (hiding). 19
4.2 RACS receiver (recovery). 20
4.3 Phone prototype: phones connect over Wi-Fi and the laptop does all the work. 22

5.1 Order of work. Every phase ends with an artifact and an automatic verification gate; nothing RACS-specific was wired in before the baseline gate passed. 24
5.2 Rerank weight sweep on calibration data only. Mixing in some structure beats pure CLIP ranking; $\alpha = 0.75$ scores best and changes the top-10 pool for two thirds of the images. 29
5.3 How the exact-recovery layer packs a secret. The token is stretched into three independent values, each used by one step; a wrong token produces a different MAC key, so the seal check fails and the file is rejected. 31
5.4 One exchange in the phone prototype. The laptop does all computation; a recompressed photo or a wrong key is detected by the seal and rejected. 32
5.5 Automatic verification checks per phase, from the stored gate records. Phases 17–21 are the RACS phases. 33

6.1 Cost of obtaining the reference image: retrieval is about 82× faster and needs no GPU. 36
6.2 Recovery PSNR versus SNR for both systems. 36
6.3 How well two classical detectors separate stego from clean carriers (0.5 = guessing, 1.0 = always detected). Textured ± 1 embedding defeats chi-square; the dithered carrier also weakens SPA. 38
6.4 Prototype timings for one photograph (hybrid mode, laptop). Bars show typical values; labels give the measured ranges. . . . 39

7.1 Proposed RACS-Hybrid: retrieval by default, generation only as a fallback. The threshold uses public scores only, so both ends make the same choice. 44

List of Tables

2.1 Comparison of related approaches with this project. 12

3.1 Functional requirements. 13
3.2 Pretrained models and their role. 16

4.1 What each party knows. 17
4.2 New RACS blocks (labels as in Figures 4.1 and 4.2) and the reason for each design choice. 21
4.3 Modules of the implementation. 23

6.1 Baseline recovery PSNR (dB) by class and SNR. 34
6.2 Recovery PSNR under the three receiver conditions (10 dB). . . . 35
6.3 Aggregate comparison (24 paired cases). 37
6.4 Outcome of the four hypotheses. 37
6.5 Exact-recovery layer, 66 secrets (each stego written to and read back from PNG). 38
6.6 Documented differences from the paper and their reason. 41

Chapter 1

Introduction

Imagine sending a photograph across a wireless network so that anyone listening sees only an ordinary picture, while the intended receiver, holding a short secret code, pulls out a completely different hidden image. That is *steganographic communication*: instead of locking a message, it hides the fact that a message exists at all.

Modern wireless systems are making this both more important and more achievable. *Semantic communication* sends the meaning of an image rather than its raw bits, which is efficient but also easy for a well-equipped eavesdropper to understand. Generative AI offers a way to hide the secret inside the semantics themselves. The recent **AgentSemSteCom** scheme [15] does exactly this: a language model describes the image, a diffusion model paints a reference picture controlled by a private token, and the secret is blended into a natural-looking stego image that travels over the channel.

The catch is cost and accuracy. Every message requires the sender *and* the receiver to run a diffusion model to paint the same reference picture, and the secret that comes out at the end is only an approximation of what went in.

1.1 Outline

This project rebuilds AgentSemSteCom from scratch on a laptop GPU and then improves it with **RACS** (Retrieval-Augmented Coverless Steganography). The key idea is simple: the reference picture does not need to be *painted*; it can be *found*. RACS looks it up in an indexed image collection, and the private token decides which candidate is used. RACS also adds an exact-recovery layer, so the receiver gets back the secret pixel for pixel. A phone app shows the whole system working on real photographs.

1.2 Problem Statement

AgentSemSteCom has two practical weaknesses:

- **Expensive references.** Generating the reference takes about 4 s and 2.6 GB of GPU memory per message, at both ends, and both ends must produce exactly the same image.
- **Approximate recovery.** The secret comes back at roughly 22 dB PSNR: recognisable, but visibly changed, which is unacceptable for documents, faces or evidence.

The question this project answers is: *can the reference be obtained cheaply and deterministically without diffusion, and can the secret be delivered exactly, without losing quality, security or stealth?*

1.3 Motivation

Retrieval has already replaced expensive generation in other areas of AI [4], and a steganographic reference only needs to look related to the secret, match its layout, be reproducible at both ends and be unpredictable without the token. A fixed image collection searched with CLIP [19] meets all four requirements at a fraction of the cost. At the same time, users of a real messaging app expect to receive *the* photo they were sent, not a reconstruction of it, which motivates exact recovery.

1.4 Purpose

The objectives of this project are to:

1. reproduce AgentSemSteCom faithfully, with every difference from the paper recorded;
2. design RACS and compare it with the baseline under identical conditions, measuring speed, memory, recovery quality, security and robustness from 5 to 20 dB SNR;
3. deliver the secret exactly, reject wrong keys and resist steganalysis;
4. demonstrate the system end to end in a phone prototype.

The rest of the report covers related work (Chapter 2), requirements (Chapter 3), the architecture of both systems (Chapter 4), implementation (Chapter 5), results (Chapter 6) and conclusions (Chapter 7).

Chapter 2

Literature Survey

This chapter reviews the work the project builds on: semantic communication, steganography and steganalysis, diffusion models, the supporting vision models, and multimodal retrieval.

2.1 Semantic Communication

Classical communication compresses an image and then protects the bits with a channel code. Deep joint source–channel coding (Deep JSCC) [2] instead trains one neural network that maps an image straight to channel symbols and another that maps noisy symbols back to an image. Quality degrades gracefully as the signal-to-noise ratio (SNR) drops, instead of collapsing suddenly. SwinJSCC [26] uses a Swin Transformer backbone and is the codec used by AgentSemSteCom. Because a semantic representation is designed to be understood, an eavesdropper with a similar decoder can understand it too, which motivates hiding private content inside it.

2.2 Steganography and Steganalysis

2.2.1 Embedding-based steganography

The classic approach changes the least significant bits (LSBs) of a cover image to carry the message. It is exact but leaves statistical traces: the chi-square attack [24] notices that LSB replacement evens out the counts of neighbouring pixel values, and Sample Pair Analysis (SPA) [5] estimates how much was embedded from pairs of adjacent pixels. Changing values by ± 1 instead of overwriting bits, and embedding only in textured regions, are the standard defences [6].

2.2.2 Deep steganography

Baluja [1] showed that a neural network can hide one whole image inside another. HiNet [9] and ISN [14] improved this with invertible networks. The AgentSemSteCom paper reports that a CNN detector catches these methods 95 %, 56 % and 56 % of the time, against 51 % (close to guessing) for its own scheme.

2.2.3 Coverless and generative steganography

Coverless methods avoid modifying a cover. Early ones mapped secret bits to existing images by hashing their features [32], with very low capacity. Generative methods create the carrier instead: DiffStega [27] uses a pretrained diffusion model and a password-dependent reference to hide an image without any training, and released the UniStega benchmark used here for calibration. AgentSemSteCom adds an LLM-written public key, token-seeded reference generation, a token-controlled perturbation and a semantic channel.

2.2.4 CNN steganalysis

XuNet [25] is a compact detector with a fixed high-pass first layer that amplifies the noise residue where hidden data lives. It is the detector used by the paper and in this project. Accuracy near 50 % on balanced data means it cannot tell stego from clean images.

2.3 Diffusion Models and Exact Inversion

A diffusion model learns to turn random noise into an image step by step, by repeatedly predicting and removing noise [7]. The DDIM sampler [21] makes this process deterministic, so it can also be run backwards to turn an image into noise. Latent diffusion [20], the basis of Stable Diffusion, runs the process in the compressed latent space of a variational autoencoder (VAE) to save computation.

Running DDIM backwards is only approximately reversible. EDICT [22] fixes this by tracking two coupled copies of the latent that update each other in turn, so every step can be undone exactly. This exactness is what lets a receiver undo the sender's hiding process. One practical detail matters later: EDICT's mixing coefficient must stay away from 0.5, where the coupling cannot be inverted.

2.4 Conditioning, Perception and Restoration Models

- **ControlNet** [29] steers a diffusion model with a spatial map such as a segmentation map or a pose skeleton.
- **IP-Adapter** [28] lets a diffusion model take an image as a prompt, in addition to text.
- **BLIP** [12] captions images; **Qwen2.5** [18] is a small language model that rewrites a caption into a prompt.
- **OneFormer** [8] segments scenes; **OpenPose** [3] finds body and face keypoints.
- **CodeFormer** [31] restores damaged faces.

2.5 Multimodal Retrieval

CLIP [19] places images and text in one shared space, so a sentence can be used to search for matching pictures. FAISS [10] provides fast nearest-neighbour search; its exact index always returns the same answer, which is essential when two parties must agree. Re-Imagen [4] showed that retrieving real images can guide or replace generation. RACS uses the retrieved image directly as the steganographic reference.

2.6 Evaluation Metrics

Recovered images are compared with the original using mean squared error (MSE), peak signal-to-noise ratio (PSNR, in dB, higher is better), structural similarity (SSIM) [23] and the learned perceptual distance LPIPS [30] (lower is better). Security is the PSNR difference between recovering with the correct token and with a wrong one. Detectability is reported as detector accuracy or as AUC, where 0.5 means the detector is guessing.

2.7 Research Gap

Table 2.1: Comparison of related approaches with this project.

Approach	**Coverless**	**Semantic channel**	**Reference**	**Exact secret**
LSB / adaptive embedding	No	No	–	Yes
Baluja, HiNet, ISN	No	No	–	No
DiffStega	Yes	No	Generated	No
AgentSemSteCom	Yes	Yes	Generated	No
RACS (this project)	**Yes**	**Yes**	**Retrieved**	**Yes**

No earlier work asks whether the generated reference of a coverless semantic steganography system can be replaced by retrieval while both ends still agree bit for bit, or combines a generative stego with an exact, authenticated payload whose detectability is measured.

Chapter 3

Requirement Engineering

This chapter states what the system must do, the qualities it must have, and the constraints it was built under. Each requirement is tied to an automatic verification check (Section 3.9).

3.1 Stakeholders

- **Sender:** holds a private image and a shared token; wants to send the image without revealing that a secret is being sent.
- **Receiver:** holds the same token; wants the secret back, and wants a wrong or tampered message to be detected rather than silently accepted.
- **Eavesdropper:** sees the channel and all public information; may try a wrong token, or try to detect that steganography is being used.
- **Researcher:** needs to compare the baseline and RACS fairly and reproduce every number.

3.2 Functional Requirements

Table 3.1: Functional requirements.

ID	Requirement
FR1	Generate a public text key K_{pub} from the secret image, deterministically.
FR2	Extract a public structure map X_{feat}: segmentation for scenes, pose skeleton for faces.

ID	Requirement
FR3	Derive every private random value from the token with SHA-256, identically at both ends.
FR4	Baseline: generate the reference X_{ref} with PicX-Real and ControlNet.
FR5	RACS: retrieve X_{ref} from a fixed corpus using only public data, rerank it, and let the token pick one of the top K.
FR6	Hide the secret with EDICT, conditioned on X_{ref}, with token noise and a token mask.
FR7	Send the stego through SwinJSCC and an AWGN channel at 5, 10, 15 and 20 dB.
FR8	Recover the secret by reproducing X_{ref} and reversing the hiding; restore faces with CodeFormer.
FR9	Test recovery with the correct, a wrong and no token; train a XuNet steganalyser.
FR10	RACS: deliver the secret file exactly through an encrypted, authenticated embedding layer, or reject it.
FR11	Provide a phone interface for sending and receiving.

3.3 Non-Functional Requirements

- **Same reference at both ends.** Sender and receiver must produce a bit-identical reference. They must agree on the corpus, models, index, ranking weights, K and all hiding parameters.
- **Fair comparison.** RACS may change only how the reference is obtained (plus the optional exact layer); every other stage is the baseline's own code.
- **Reproducibility.** Every result file records the configuration, data and model hashes.
- **Honest claims.** Every difference from the paper is recorded with its reason and impact; calibration data never appears in an evaluation table.
- **Efficiency and locality.** Fit in 6 GB of GPU memory; no cloud model in the sending or receiving path.

3.4 Hardware Requirements

The whole system runs on a single laptop, which is itself a requirement: RACS should not need a server-class GPU.

- **GPU:** NVIDIA GeForce RTX 3050 Laptop with 6 GB of VRAM.
- **Memory and storage:** 16 GB RAM; SSD, with models and data on a secondary drive.
- **Operating system:** Windows 11.
- **Prototype clients:** two smartphones on the same Wi-Fi network as the laptop.

3.5 Software Requirements

- **Core:** Python 3.10 and PyTorch 2.6 with CUDA 12.4.
- **Models:** Hugging Face Diffusers and Transformers for Stable Diffusion, ControlNet, IPAdapter, BLIP, Qwen2.5, OneFormer and CLIP; the OpenPose annotator; and the official SwinJSCC and CodeFormer code.
- **Retrieval and metrics:** FAISS for the exact index; NumPy, SciPy and Pillow for numerics and images; the LPIPS package for the perceptual metric.
- **Cryptography:** the Python standard library (`hashlib`, `hmac`) for SHA-256, SHAKE-256, PBKDF2 and HMAC, so no extra security dependency is needed.
- **Project tooling:** one YAML configuration file read by both ends, and Git for version control.

3.6 Models Used

Table 3.2 lists the pretrained model behind each stage of the pipeline.

3.7 Datasets

- **Retrieval corpus:** 1,000 COCO [13] images at 512×512, with a versioned index whose row order is fixed.
- **Calibration split:** 22 images per class from UniStega [27], used only to tune and to sanity-check against the paper.

Table 3.2: Pretrained models and their role.

Stage	**Model**
Image caption	BLIP (base)
Public key K_{pub}	Qwen2.5-0.5B-Instruct
Structure (scenes)	OneFormer, ADE20K, Swin-L
Structure (faces)	OpenPose body and face
Reference (baseline)	PicX-Real + SD v1.5 ControlNet
Reference (RACS)	CLIP ViT-B/32 + exact index
Image prompt	IP-Adapter-plus
Hiding	EDICT on Stable Diffusion v1.5
Channel	SwinJSCC, trained at 10 dB
Face restoration	CodeFormer
Steganalysis	XuNet (trained in this project)

- **Evaluation split:** 22 images per class, never seen during tuning: COCO scenes (common), Wikimedia Commons portraits (facial) and cat photographs (style).

3.8 Constraints

- **6 GB GPU:** the pipeline runs at 256 px; large encoders run on the CPU.
- **Host memory:** large model files needed a special low-memory loader on Windows.
- **Library conflicts:** FAISS and PyTorch cannot share a process on Windows, so retrieval runs in its own process.
- **Unavailable component:** the paper's digital codec branch has no public weights and is not reproduced.

3.9 Verification Gates

Every phase ends with a script that writes named pass/fail checks to a JSON file, for example "the VAE round trip is measured", "the same token always picks the same image", and "the sender and receiver reference are identical for every test image". A check can be changed only with a written reason, never quietly relaxed.

Chapter 4

System Design

This chapter presents the architecture of RACS from the sender's and the receiver's side. In the diagrams, grey blocks are components inherited unchanged from the baseline (AgentSemSteCom) and green blocks are new in RACS.

4.1 Information Model

Table 4.1: What each party knows.

Symbol	**Visibility**	**Meaning**
X_s	Private	The secret image
K_{pub}	Public	A text description of the image, written by a language model
X_{feat}	Public	The image's structure: a segmentation map or a pose skeleton
τ	Shared secret	An 8-digit token, the only secret sender and receiver share
X_{ref}	Derived	A reference image that both ends must reproduce identically
X_{stego}	Transmitted	The innocent-looking image that is actually sent

Every private random value (seeds, noise, mask, selection) is derived from τ with SHA-256, so both ends compute exactly the same values on different machines.

4.2 Starting Point: the Baseline

In AgentSemSteCom, the sender describes the secret in words (K_{pub}) and extracts its structure (X_{feat}); a diffusion model then paints a reference image X_{ref} from these and a token-seeded noise; EDICT blends the secret toward that reference to create the stego image, which is sent through the Swin-JSCC semantic codec over a noisy channel. The receiver paints the same reference again and runs EDICT backwards. This costs about 4 s and 2.6 GB of GPU memory per reference at *each* end, and the secret comes back only approximately.

4.3 RACS Architecture

RACS keeps everything that works in the baseline and changes two things: the reference image is *retrieved* instead of generated, and an exact-recovery layer carries the secret losslessly (Path B) next to the semantic channel (Path A). Figures 4.1 and 4.2 show the sender and the receiver. Green blocks are new, grey blocks are the baseline's own unchanged code, and the dashed red block is the baseline stage that RACS removes.

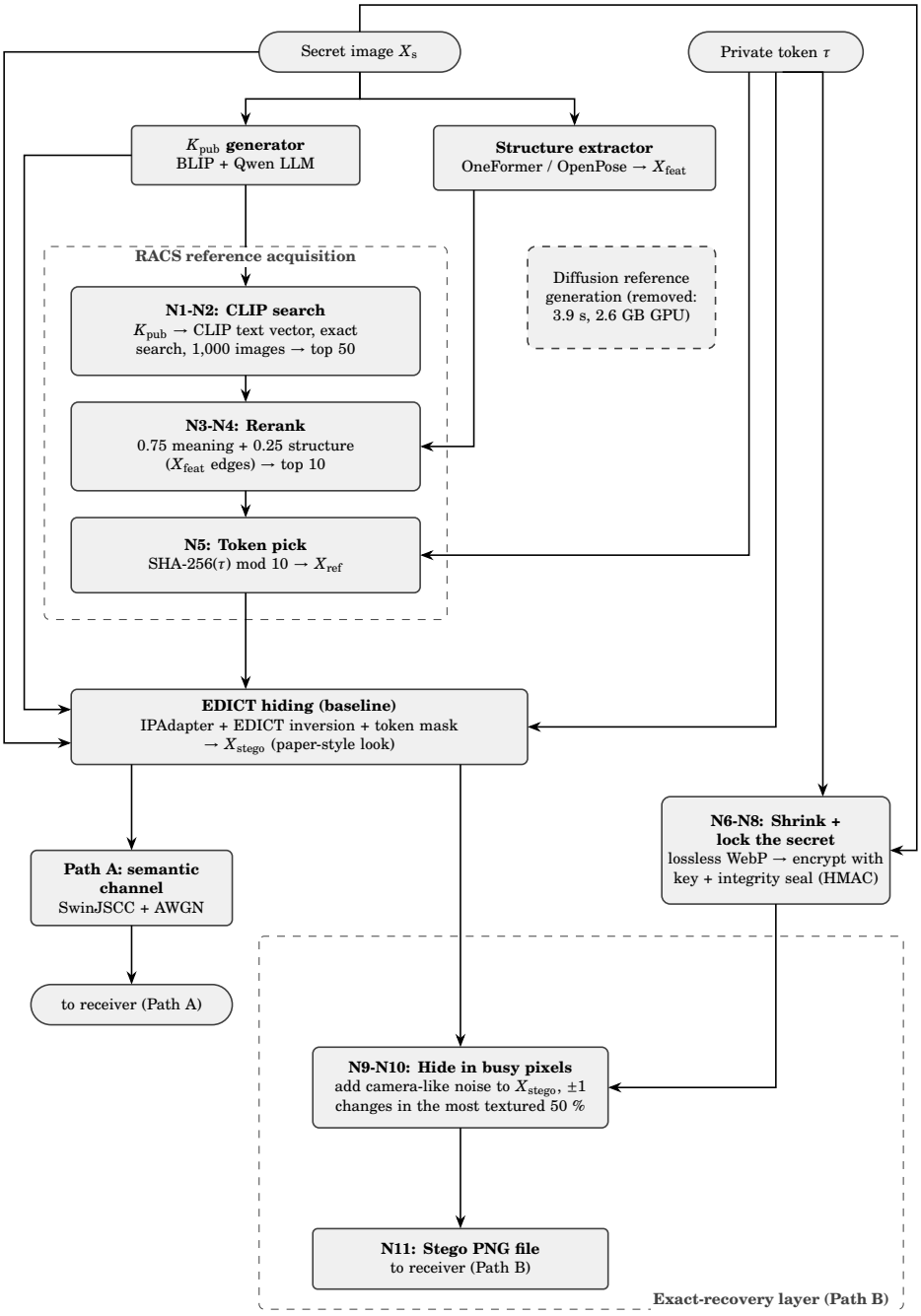


Figure 4.1: RACS sender (hiding).

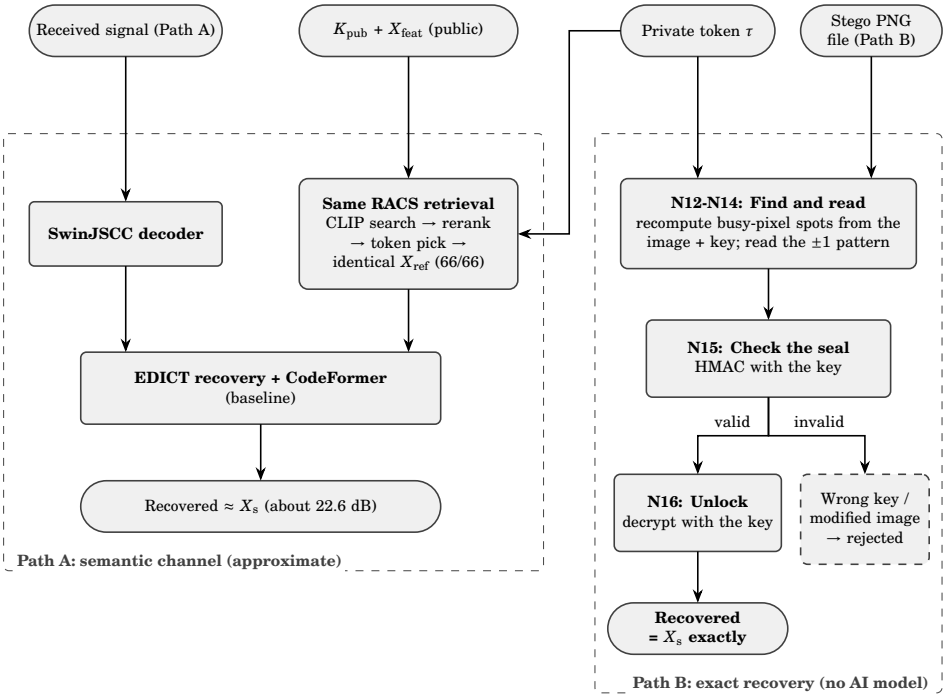


Figure 4.2: RACS receiver (recovery).

The reference is found, not painted. The public key K_{pub} is turned into a CLIP embedding and used to search a fixed collection of 1,000 images. The 50 closest images are re-scored by how well they match in meaning (75 %) and in layout (25 %, comparing edge maps with the public structure X_{feat}), and the token picks one of the best 10. The receiver repeats exactly the same steps from public data and the token and gets exactly the same image (66 of 66 test images), in 0.05 s and without a GPU.

The secret arrives exactly. The stego image still travels through the semantic channel (Path A), giving an approximate recovery as in the baseline. In addition, the original secret is compressed losslessly, encrypted and sealed with a key derived from the token, and hidden with tiny ±1 changes in the most textured pixels of the stego image, which is sent as a PNG file (Path B). The receiver recomputes the same pixel positions, checks the seal and decrypts. A wrong key or an altered file is rejected instead of producing a misleading image.

4.4 New-Block Reference

One change at a time. In the code, RACS is a subclass of the baseline that overrides a single method: where the reference comes from. Every other

Table 4.2: New RACS blocks (labels as in Figures 4.1 and 4.2) and the reason for each design choice.

Block	**What it does**	**Design choice and reason**
N1	CLIP text embedding of K_{pub}	Query from *public* data only, so the receiver can recompute it without the secret.
N2	Exact search, top-50 pool	No approximate index, so sender and receiver get identical rankings.
N3	Edge-map structural score	CLIP finds the right content but not the right layout; X_{feat} is the only public structure both sides hold.
N4	0.75 / 0.25 rerank, top 10	Weights chosen on calibration images only; reranking a wider pool changes *which* images the token can pick.
N5	SHA-256(τ) mod K	One implementation at both ends; a 1-in-K guess for an attacker.
N6	Lossless WebP compaction	32 % smaller payload, which makes a low embedding rate affordable.
N7	PBKDF2 key derivation [16]	Slows brute force of the short numeric token.
N8, N16	Stream encryption [17]	Hidden bits look random.
N8, N15	HMAC-SHA256 seal [11]	A wrong key or modified image is *rejected*, never shown as a wrong image.
N9	Carrier upscale + dither	An upscaled image has an unnaturally clean LSB plane; camera-like noise makes it realistic.
N10, N13	Texture-adaptive ± 1	Changes only busy regions; ± 1 instead of overwriting defeats the chi-square attack; the receiver recomputes positions with no side information.
N11, N12	PNG file transfer	Lossless; recompression would destroy the payload (and is detected).

stage is imported, not copied, so any measured difference comes from the reference alone.

4.5 Prototype Architecture

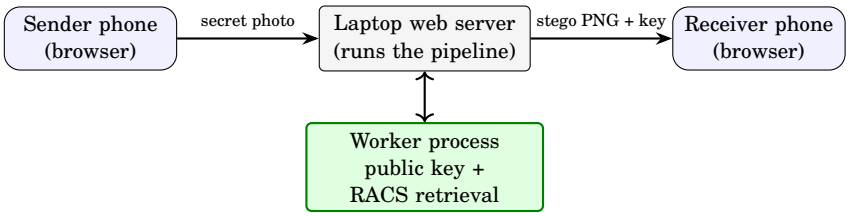


Figure 4.3: Phone prototype: phones connect over Wi-Fi and the laptop does all the work.

The prototype offers three modes: *hybrid* (default: diffusion stego look plus the exact layer), *lossless* (exact layer on a retrieved cover, no diffusion) and *diffusion* (the paper's approximate pipeline). The stego must be sent as a file, because messaging apps recompress photos; the receiver detects this through the seal.

4.6 Code Organisation

Table 4.3: Modules of the implementation.

Package	**Responsibility**
`common`	Paths, seeding, device selection, low-memory model loading
`configs`	One configuration file read by both ends
`semantic`	Public key and structure extraction
`reference`	Baseline reference generation and ControlNet fine-tuning
`retrieval`	RACS: corpus, search, structure cache, rerank, token pick
`steganography`	EDICT, hiding, recovery, token mask, IPAdapter
`communication`	AWGN channel and SwinJSCC codec
`enhancement`	CodeFormer face restoration
`pipelines`	The assembled baseline and RACS systems
`evaluation`	Metrics, frozen test splits, reports
`security`	Token tests, XuNet, chi-square and SPA detectors
`app`	Phone web server and the exact-recovery layer
`scripts`	Build and verification script for every phase

Chapter 5

Implementation

The system was built incrementally in phases. Each phase had a stated goal, a small experiment, an artifact saved to disk and a verification gate (Section 3.9). This chapter describes the phases in order, then the engineering problems that shaped the implementation.

5.1 Methodology

The working rule was *baseline first*: no RACS component was wired into the pipeline until the reproduced baseline passed its gate, so the comparison in Chapter 6 is against a measured baseline rather than against numbers copied from the paper. Every phase followed the same loop: state what is built and why; build the smallest experiment; test it; explain the output; integrate; save the artifact with its configuration hash; move on.

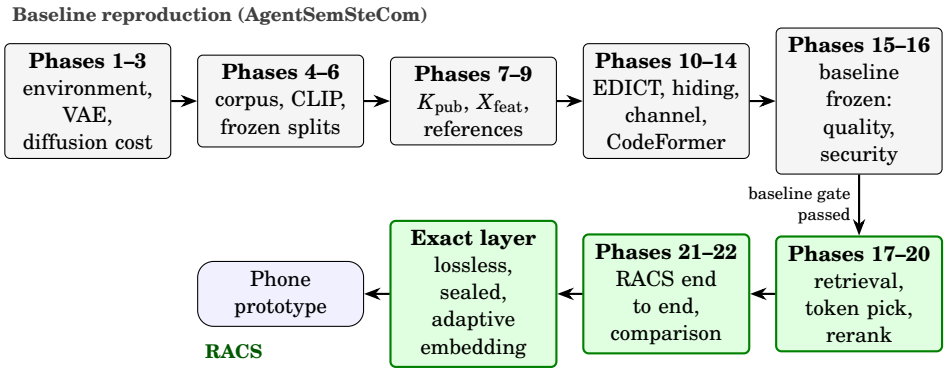


Figure 5.1: Order of work. Every phase ends with an artifact and an automatic verification gate; nothing RACS-specific was wired in before the baseline gate passed.

5.2 Foundation (Phases 1–3)

5.2.1 Phase 1: environment and tensor contract

A virtual environment with PyTorch 2.6 and CUDA 12.4 was created, and all model caches were redirected to the data drive. A seeding utility fixes Python, NumPy and PyTorch (CPU and CUDA) random states. The tensor contract was fixed: images are RGB, `float32`, shape $(1,3,H,W)$, values in $[0,1]$ for metrics and $[-1,1]$ for the VAE. The gate checks CUDA availability, configuration loading, tensor shapes and that two seeded runs produce identical tensors.

5.2.2 Phase 2: VAE and latent space

The Stable Diffusion VAE compresses a $512 \times 512 \times 3$ image to a $64 \times 64 \times 4$ latent (an $8\times$ spatial reduction). A round trip over five test images gave a mean PSNR of 32.81 dB (range 31.78–34.15) and SSIM 0.935. This sets a ceiling: any pipeline that passes through the VAE cannot recover a secret better than about 33 dB.

5.2.3 Phase 3: diffusion generation cost

Stable Diffusion v1.5 was loaded in half precision with attention slicing and a DDIM scheduler. A sweep over steps and seeds measured cost: at 20 steps one 512 px image took 5.56 s (0.278 s per step) with 2.60 GB peak GPU memory, so the paper's 50 steps cost 13.5 s per image. Disabling classifier-free guidance reduced cost by a factor of 1.73, confirming that the UNet forward pass dominates. Cost grows linearly with the number of denoising steps (5.56, 8.18 and 13.47 s at 20, 30 and 50 steps), which is exactly the per-message cost that RACS removes.

5.3 Retrieval Inputs and Test Harness (Phases 4–6)

5.3.1 Phase 4: retrieval corpus

A builder deterministically sampled 1,000 images from COCO val2017 with a fixed seed, centre-cropped and resized them to 512×512 RGB, and wrote a versioned index file listing each image identifier, path and SHA-256 hash. The row order of this file is part of the protocol, because the index returns row numbers. The verification gate checks every hash, the row count and that a rebuild is identical.

5.3.2 Phase 5: CLIP embeddings

CLIP ViT-B/32 embeds each corpus image to a 512-dimensional, L2-normalised `float32` vector. Embeddings are kept in single precision because half preci-

sion changed nearest-neighbour orderings in borderline cases. Re-encoding the corpus gave a maximum absolute difference of 2.5×10^{-7}. Using COCO captions as queries, the correct image was the top-1 result 47.6 % of the time out of 1,000; mean similarity was 0.779 within a topic and 0.542 across topics.

An important observation was that CLIP similarity and structural similarity disagree: two images can be semantically close but laid out completely differently (in one case, CLIP similarity 0.643 but edge overlap only 0.199). This motivated the structural reranking of Phase 20.

5.3.3 Phase 6: evaluation harness and frozen splits

Two frozen, role-tagged splits were built (Section 3.7). The calibration split (UniStega, 22 per class) may only be used to check the paper's regime and to tune parameters; the evaluation split (COCO, Commons portraits, Commons cats, 22 per class, disjoint from the corpus) is the only source of claims. The split module refuses to return evaluation data to a tuning routine. Each image is assigned a fixed token stored in a versioned token file. The facial evaluation images were filtered by title and by CLIP to keep only real single-face photographs. Metrics (MSE, PSNR, SSIM, LPIPS with an AlexNet backbone) and a frozen report schema were implemented, and a self-test with 83 checks verifies them.

5.4 Baseline Reproduction (Phases 7–16)

5.4.1 Phase 7: public conditions

For each of the 132 split images, BLIP produces a perception caption with greedy decoding, and Qwen2.5-0.5B-Instruct rewrites it into a concise generation prompt using a fixed one-shot chat template. The reply is sanitised (instruction echoes, markdown and URLs removed); a degenerate reply falls back deterministically to the perception caption, and the number of fallbacks is recorded (zero occurred). OneFormer produces 150-class ADE20K segmentation for common images. OpenPose produces body and face keypoints for portraits; on 3 of the 22 tightly cropped evaluation portraits OpenPose detects no body, and a MediaPipe face mesh is used instead, with the method recorded per image. To fit in memory, the build runs in two passes (all prompts, then all features) with one model resident at a time.

5.4.2 Phase 8: reference generation

PicX-Real with the Stable Diffusion v1.5 segmentation and OpenPose ControlNets generates one reference per image at 256 px with 20 DDIM steps and guidance 7.5, from a token-derived seed. Two independent runs produced byte-identical images for all 132 references (peak GPU memory 2.84 GB). The

IPAdapter-plus image embedding of each reference is precomputed on the CPU and cached on disk keyed by the reference's SHA-256, so both ends read an identical tensor and the 2.5 GB image encoder never has to share the GPU with Stable Diffusion.

5.4.3 Phase 9: ControlNet zero-convolution fine-tuning

Algorithm 2 of the paper fine-tunes only the zero-convolution layers of ControlNet. A short run (50 steps, batch 1, `bfloat16`, calibration data only) was performed to validate the code path; half precision produced NaN gradients within five steps, while `bfloat16` was stable. A held-out probe loss improved from 0.1488 to 0.1296.

5.4.4 Phase 10: EDICT

EDICT was implemented from the coupled-transformation equations as its own module and tested in isolation by inverting and regenerating random and image latents across a sweep of step counts and mixing coefficients. At the operating point (20 steps, $p = 0.93$, half-precision UNet with single-precision latents) the maximum relative latent error was 3.6×10^{-3}: a numerical floor from half precision, not an algorithmic failure. The sweep also exposed a latent bug: the configuration included $p = 0.5$, at which the coupling is singular, so a full rebuild could never finish. The sweep now starts at 0.55.

5.4.5 Phase 11: local hiding and recovery

A full inversion treats the clean secret latent as pure noise and decodes to a reference-like image (11.3 dB against the secret), which is not imperceptible. Hiding therefore uses a bounded-strength EDICT chain (strength 0.2, 20 steps, guidance 3.0) that is still an exact paired inverse. Token noise and the sparse token mask are applied in the latent. IPAdapter injects the reference embedding at both ends, making the stego depend on the reference pixels.

5.4.6 Phase 12: AWGN channel

The channel adds Gaussian noise whose strength is set from the signal power and the target SNR, split equally across the in-phase and quadrature parts of complex symbols. A seeded generator provides the noise realisation so that experiments are repeatable. The measured noise variance matched the analytic value within 0.46 %.

5.4.7 Phase 13: semantic codec

The official SwinJSCC release with bandwidth $C = 64$ trained at 10 dB (the nearest public checkpoint to the paper's $C = 48$) was integrated in single precision, with only its noise generator replaced by a seeded one. In isolation it reconstructs images at 32.54 dB at 10 dB SNR, compared with 22.88 dB for the VAE-based substitute used before the official weights were obtained.

5.4.8 Phase 14: CodeFormer enhancement

The official CodeFormer architecture and checkpoint run at 512 px with fidelity 0.5 on the facial class only, on the CPU because of GPU memory. The gate requires measured deltas rather than assuming improvement; CodeFormer may lower PSNR slightly while improving LPIPS.

5.4.9 Phase 15: assembled baseline

The stages were assembled into one baseline object. A calibration run (15A) checks the regime against the paper's Table II with tolerances (Gate A), and an evaluation run (15B) produces the frozen table (Gate B). The manifest records the configuration hash, split and token versions, and the hash of every model. Results are in Chapter 6.

5.4.10 Phase 16: security and steganalysis

Recovery is measured under three receiver conditions: the *correct* token, a valid *wrong* token, and *no token* (a fixed public fallback token with an all-zero mask). A XuNet steganalyser with a fixed SRM high-pass first layer is trained to separate clean images from stego images, with the train, validation and test splits grouped by image so that patches of one image never cross splits.

5.5 RACS Reference Retrieval (Phases 17–22)

All retrieval logic lives in one module shared by transmitter and receiver.

5.5.1 Phase 17: exact retrieval

The corpus embeddings are indexed with an exact inner-product index. A FAISS `IndexFlatIP` is built and saved in its own process; within the pipeline, the same exact inner product is computed with NumPy, because FAISS and PyTorch load conflicting OpenMP runtimes in one process on Windows. The gate verifies that FAISS and the in-process search return identical top-50 rows and scores, that each corpus image retrieves itself first with score 1.0, and that the row order matches the index file.

5.5.2 Phase 18: token selection

The token is hashed with SHA-256 together with a fixed label, and the result modulo K gives the position of the chosen image in the top-K list. The gate verifies identical output in separate processes, that $K = 1$ always selects index 0, that over the evaluation tokens all ten buckets are used and the distribution is near uniform, and that the language's built-in hash is never called.

5.5.3 Phase 19: structural representation

Each corpus image is converted to a dilated binary edge map at 128 px and cached. Query structure is derived from the public X_{feat} (segment boundaries or keypoint coverage). The cache is deterministic, and the gate requires at least three cases in which the CLIP ranking and the structural ranking disagree; five were found.

5.5.4 Phase 20: reranking

Each candidate gets a combined score: a weight α times its CLIP similarity plus a weight β times its edge-map overlap with the public structure. A sweep on calibration data only measured, for the selected top-10 pool, the mean semantic similarity to the secret, the mean edge IoU, and the fraction of images whose pool changed relative to CLIP alone (Figure 5.2). The objective weights semantic similarity and edge IoU equally.

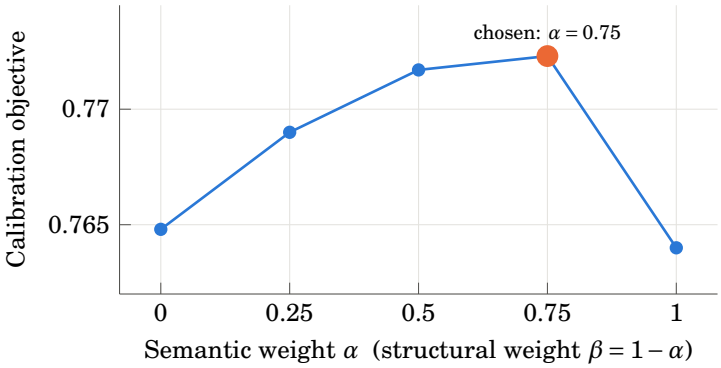


Figure 5.2: Rerank weight sweep on calibration data only. Mixing in some structure beats pure CLIP ranking; $\alpha = 0.75$ scores best and changes the top-10 pool for two thirds of the images.

$\alpha = 0.75$, $\beta = 0.25$ maximises the objective; it keeps most of the semantic quality of CLIP alone while raising structural agreement, and it changes the candidate pool for two thirds of the images. These weights were frozen before any evaluation run.

5.5.5 Phase 21: RACS end to end

RACS-Base subclasses the baseline and overrides only reference acquisition. Retrieval for the transmitter and for the receiver run in two separate processes before the diffusion stack loads (CLIP text encoding crashed when loaded beside it). The gate requires $X^{\mathrm{tx}}_{\mathrm{ref}} = X^{\mathrm{rx}}_{\mathrm{ref}}$ for every evaluation image, compared by identifier and file hash. Both systems are then measured through the same object and the same evaluation function, differing only in where the reference comes from.

5.5.6 Phase 22: controlled comparison

A comparison script reads both systems' measurements on the same cases and tokens, refuses calibration data, and reports quality, security, latency and memory side by side (Chapter 6).

5.6 RACS Exact-Recovery Layer

The layer is a self-contained module with four functions: compaction, carrier preparation, embedding and extraction. A container format stores a magic value, the length and the format of the payload, and the HMAC tag. Evaluation generates 66 paper-style carriers (EDICT at strength 0.5 with a RACS-retrieved reference), embeds every evaluation secret with four progressively stronger variants, writes each stego to a real PNG file and reads it back before extraction. Two classical detectors were implemented: Westfeld's chi-square test and Sample Pair Analysis. For each detector the AUC is computed between each stego and the same carrier without payload, so 0.5 means the detector cannot tell them apart.

Figure 5.3 shows how one secret becomes the hidden bit stream. The token is stretched once by PBKDF2 into three independent values: an encryption key, a MAC key and the seed that orders the embedding positions. Because the positions are chosen from texture computed on the top five bits of each value, and a ± 1 change never alters those bits, the receiver recomputes exactly the same positions from the stego image alone.

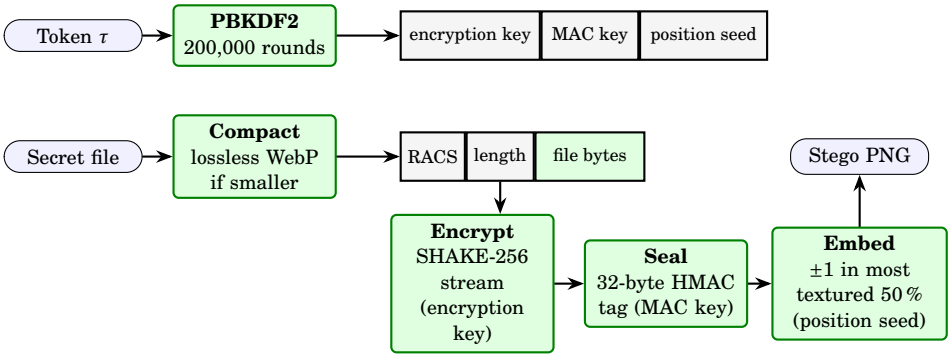


Figure 5.3: How the exact-recovery layer packs a secret. The token is stretched into three independent values, each used by one step; a wrong token produces a different MAC key, so the seal check fails and the file is rejected.

5.7 Prototype

A web server on the laptop serves a mobile page. The sender uploads a photograph and receives a stego PNG and an 8-digit key; the receiver uploads the PNG and the key and receives the secret. Photographs keep their aspect ratio by edge-padding to a square at 512 px and cropping back on receipt. The public-key generation and retrieval run in a separate worker process for the reasons given above. A self-test mode runs a round trip from the command line. Figure 5.4 shows one complete exchange.

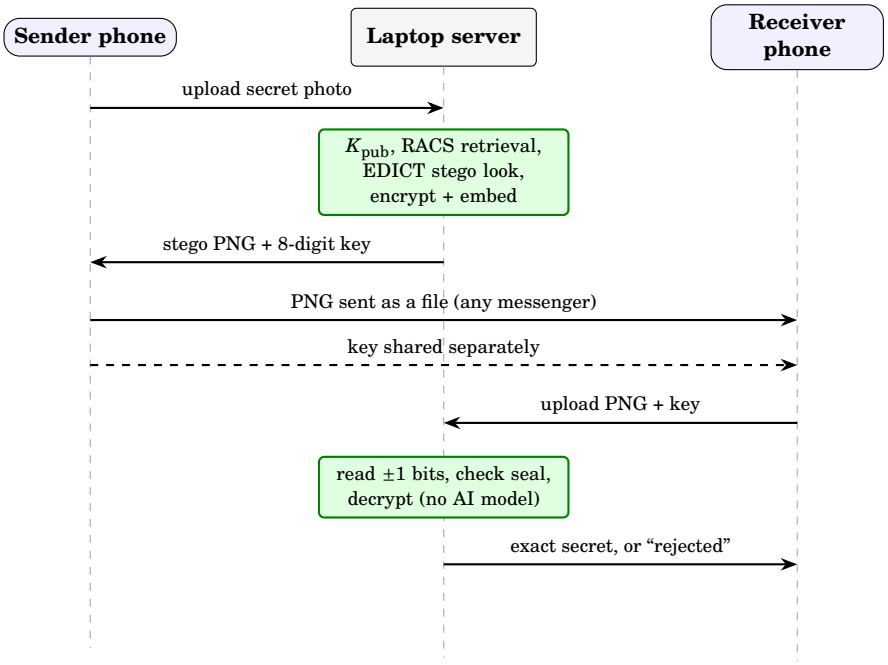


Figure 5.4: One exchange in the phone prototype. The laptop does all computation; a recompressed photo or a wrong key is detected by the seal and rejected.

5.8 Verification Results

Every phase writes its gate result to a JSON record of named checks, so the state of the whole build can be audited at any time. Figure 5.5 summarises the stored records: 119 checks across twelve phases, 113 of which pass. All six failures sit in Phases 15 and 16 and have a single cause: those phases were run on the 2-image-per-class subset, while their gates expect the full 22 images per class (sample counts, record counts, and the hash of the full frozen table). Every determinism, provenance and component check passes, including all RACS phases.

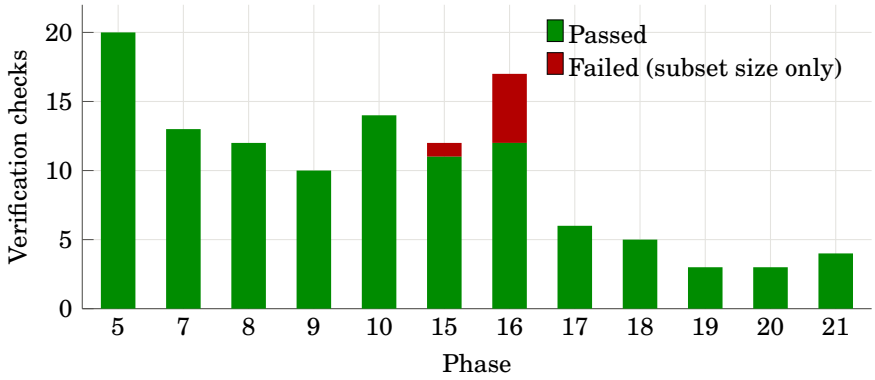


Figure 5.5: Automatic verification checks per phase, from the stored gate records. Phases 17–21 are the RACS phases.

5.9 Engineering Challenges

- **Windows commit charge.** Memory-mapped checkpoint loading reserved more virtual memory than the paging file allowed, causing access violations. A small shim loads `safetensors` files read-only before any model library is imported.
- **6 GB GPU.** Half precision, attention slicing, 256 px operation, one model resident at a time, CPU placement of the IPAdapter encoder and CodeFormer, and on-disk caches of derived tensors kept every phase within budget.
- **Cross-process determinism.** Process-randomised hashing, half-precision encoders and implicit device nondeterminism were removed from every path that affects the reference.
- **Float-to-integer truncation.** A conversion on the transmitter side truncated instead of rounding before the IPAdapter embedding, which could have desynchronised the two ends; caching by reference hash removed the risk.
- **Mixed-state detection.** Upgrading components invalidated earlier frozen artifacts. Configuration and model hashes in every manifest made the verification gates fail loudly until the affected phases were rebuilt, instead of silently mixing results.

Chapter 6

Results

This chapter reports measured results. All numbers come from the evaluation split unless labelled *calibration*; paper values are shown only as anchors and are never mixed into measured rows.

6.1 Experimental Setup

- **Classes.** Common (COCO scenes), facial (portraits), style (cat photographs).
- **Channel.** SwinJSCC ($C = 64$) over AWGN at 5, 10, 15 and 20 dB SNR.
- **Hiding.** EDICT, 20 steps, strength 0.2, mixing 0.93, $\eta = 0.05$, IPAdapter-plus weight 1, 256 px.
- **Sample sizes.** The baseline and the RACS comparison were measured on 2 images per class at all four SNRs (24 cases per system); the transmitter–receiver invariant and the exact-recovery layer were measured on all 66 evaluation images.

6.2 Baseline Recovery Quality

Table 6.1: Baseline recovery PSNR (dB) by class and SNR.

Class	**5 dB**	**10 dB**	**15 dB**	**20 dB**
Common	18.30	20.72	21.53	21.45
Facial	22.56	24.53	24.89	25.10
Style	19.93	22.68	23.28	23.43

Quality rises with SNR and flattens above 15 dB, where the channel stops being the bottleneck and the VAE and diffusion round trip limit quality. The perceptual metrics follow the same pattern: from 5 to 20 dB, SSIM improves from 0.42–0.60 to 0.58–0.72 and LPIPS falls from 0.23–0.46 to 0.14–0.24 across the three classes. Faces score highest, helped by CodeFormer. Against the paper's Table II anchors, the facial class at 5 dB reaches 22.56 dB (paper 22.73) and at 20 dB 25.10 dB with LPIPS 0.140 (paper 26.29 dB, LPIPS 0.213); the common class at 5 dB reaches 18.30 dB (paper 19.71); the style class at 5 dB has LPIPS 0.345 (paper 0.385). Absolute PSNR is about 1–3 dB below the paper, consistent with the reduced resolution, step count and strength forced by the hardware (Section 6.8).

6.3 Baseline Security and Steganalysis

Table 6.2: Recovery PSNR under the three receiver conditions (10 dB).

Condition	**This work**	**Paper (approx.)**
Correct token	22.37 dB	25.37 dB
Wrong token	13.96 dB	18.91 dB
No token	14.02 dB	–
Correct − wrong gap	**8.41 dB**	6.46 dB
XuNet detection accuracy	0.500	0.509

A wrong or missing token reduces recovery to about 14 dB, an unrecognisable image. The gap between correct and wrong token is larger than in the paper, so a wrong key reveals less. XuNet is at chance, so this detector cannot distinguish stego images from clean ones.

6.4 RACS versus the Baseline

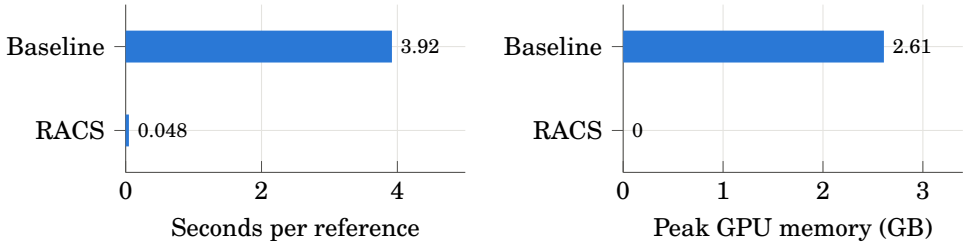
6.4.1 Protocol invariant

In separate transmitter and receiver processes, RACS selected the same reference for **66 of 66** evaluation images, with identical file hashes. The FAISS index and the in-process exact search returned identical top-50 rows and scores.

6.4.2 Efficiency

Obtaining the reference is the stage RACS replaces, and it is where the difference is largest (Figure 6.1). Diffusion generation takes 3.92 s and 2.61 GB

of GPU memory per reference; retrieval takes 0.048 s (0.51 s including loading the image from disk) and needs no GPU at all, about 82× faster. Because both ends must obtain the reference for every message, the saving applies twice per message.



(a) Reference latency.

(b) Reference GPU memory.

Figure 6.1: Cost of obtaining the reference image: retrieval is about 82× faster and needs no GPU.

6.4.3 Quality and security

Both systems were measured on the same cases, tokens and code path. Figure 6.2 shows recovery PSNR for every class and SNR: the RACS curve (solid) lies on or slightly above the baseline curve (dashed) in 11 of the 12 cells, with the largest gain for faces (up to +0.64 dB).

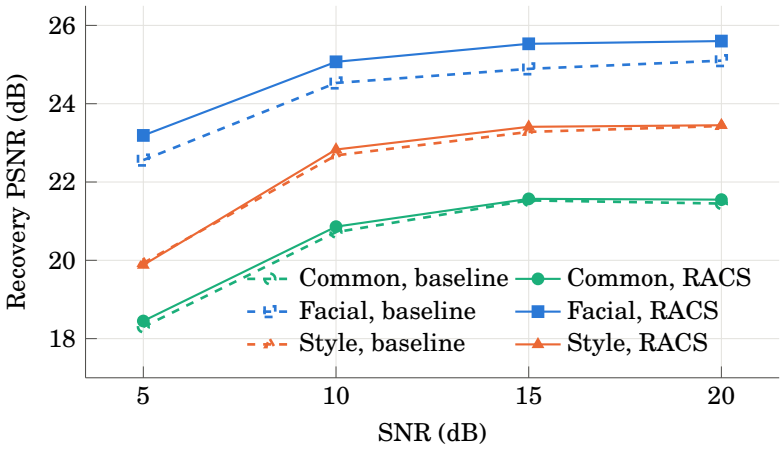


Figure 6.2: Recovery PSNR versus SNR for both systems.

Table 6.3: Aggregate comparison (24 paired cases).

Metric	**Baseline**	**RACS**
PSNR (dB) ↑	22.37	**22.62**
SSIM ↑	0.616	**0.627**
LPIPS ↓	0.238	**0.227**
Correct-token PSNR, 10 dB	22.64	22.92
Wrong-token PSNR, 10 dB	13.97	13.98
No-token PSNR, 10 dB	14.09	14.08
Wrong token and wrong reference, 10 dB	–	13.98
Security gap (correct − wrong), 10 dB	8.67 dB	**8.94 dB**

The paired mean PSNR difference is +0.25 dB in favour of RACS, with SSIM and LPIPS also slightly better and the security gap preserved. A plausible explanation is that a retrieved photograph is a natural image whose statistics suit the IPAdapter conditioning better than a generated 256 px image does. With 2 images per class this difference is indicative rather than statistically established; the important finding is that replacing generation with retrieval does *not* degrade quality or security while removing the GPU cost of the reference.

6.4.4 Hypotheses

Table 6.4: Outcome of the four hypotheses.

Hypothesis	**Evidence**	**Outcome**
H1 Efficiency	82× faster reference, no GPU memory	Supported
H2 Quality	PSNR, SSIM, LPIPS equal or better	Supported
H3 Security	Gap 8.94 vs 8.67 dB	Supported
H4 Robustness	Same trend across 5–20 dB, all classes	Supported

6.5 RACS Exact-Recovery Layer

Table 6.5 compares four variants on all 66 evaluation secrets. Variant A writes the original file into two LSBs; B adds lossless compaction; C replaces LSB writing with texture-adaptive ±1 embedding; D (proposed) adds the dithered carrier.

Table 6.5: Exact-recovery layer, 66 secrets (each stego written to and read back from PNG).

Variant	Exact	Wrong key rejected	Payload (KB)	Bits per value	PSNR (dB)	Chi-square AUC	SPA AUC
Generative only	0 %	–	–	–	–	–	–
A: naive 2-LSB	100 %	100 %	386	2.00	44.1	1.00	1.00
B: + compression	100 %	100 %	261	2.00	44.1	1.00	1.00
C: + textured ±1	100 %	100 %	261	0.25	57.2	**0.50**	1.00
D: + dither (RACS)	**100 %**	**100 %**	**261**	**0.25**	**57.2**	**0.50**	**0.83**

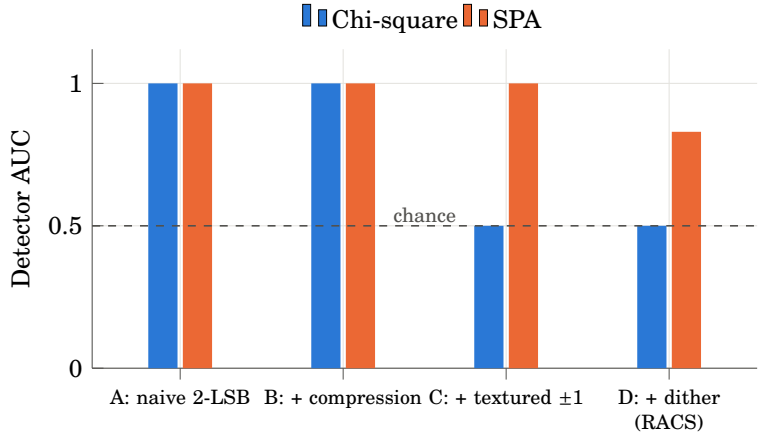


Figure 6.3: How well two classical detectors separate stego from clean carriers (0.5 = guessing, 1.0 = always detected). Textured ±1 embedding defeats chi-square; the dithered carrier also weakens SPA.

- **Exactness.** All 66 secrets are recovered pixel-identical by every variant, against 22.37 dB approximate recovery from the generative pipeline alone.
- **Authentication.** All 66 wrong keys are rejected by the MAC; no plausible wrong image is ever produced.
- **Compaction** reduces the payload by 32 % (386 to 261 KB), which makes a low embedding rate affordable.
- **Chi-square is defeated** by adaptive ±1 embedding (AUC 1.00 to 0.50).
- **SPA is weakened but not defeated.** The estimated embedding rate falls from 0.40 to 0.09, and the AUC falls from 1.00 to 0.83 only once the carrier carries realistic sensor noise; without the dither SPA remains at 1.00 even at 0.25 bits per value. The cover source matters as much as the embedding.

- **Distortion** against the payload-free carrier is 57.2 dB PSNR, compared with 44.1 dB for naive embedding.
- **Cost** is about 2.5 s to embed and 2.2 s to extract on the CPU.

6.6 Prototype Performance

Figure 6.4 breaks down one exchange for a 902×1600 photograph in the default hybrid mode. Sending is dominated by the two model stages that give the stego its natural look (writing the public key and the EDICT pass); the RACS reference costs half a second. Receiving needs no AI model at all and returns a pixel-identical secret in one to two seconds.

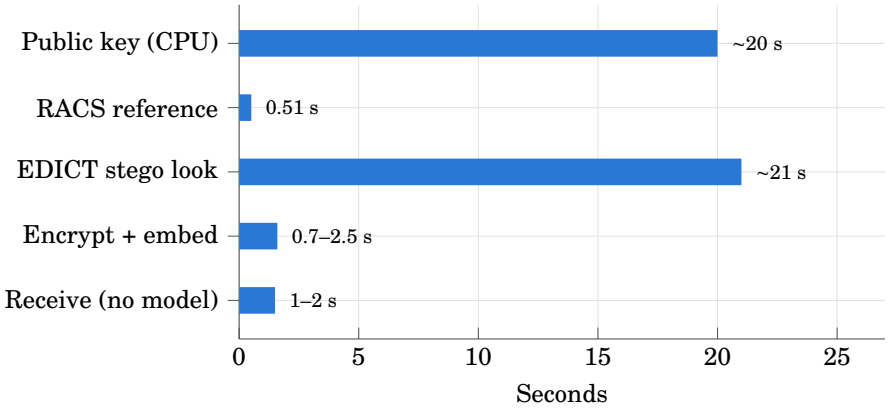


Figure 6.4: Prototype timings for one photograph (hybrid mode, laptop). Bars show typical values; labels give the measured ranges.

6.7 Discussion

6.7.1 Why retrieval does not cost quality

One might expect a generated reference, built specifically from the secret's own description and structure, to beat a reference picked from a fixed collection. The measurements say otherwise. The reference enters the hiding process only through the IPAdapter image embedding at bounded EDICT strength, so what matters is that it is semantically close and natural-looking, not that it is pixel-aligned with the secret. A real photograph satisfies this at least as well as a 256 px generated image, and the structural rerank keeps the layout roughly compatible. The small, consistent gain for faces suggests that real portraits from the corpus suit the IPAdapter conditioning slightly better than synthetic ones.

6.7.2 Security of the token

The private token does three jobs in RACS: it picks the reference, it seeds the noise and mask in EDICT, and it derives the keys of the exact layer. An attacker who knows all public information can rebuild the top-10 candidate pool and so guess the right reference with probability 1/10; but with the wrong token the noise and mask are still wrong, and the measured recovery with a wrong token *and* a wrong reference is the same 13.98 dB as with a wrong token alone. For the exact layer, every guess of the 8-digit token costs 200,000 PBKDF2 iterations, so trying all 10^8 tokens costs 2×10^{13} HMAC evaluations, and every wrong guess is rejected by the seal rather than producing a plausible image that could leak information. A longer key would raise this cost further at no extra cost to legitimate users.

6.7.3 When RACS may not fit

Retrieval can only return what the corpus contains. For a secret far from every corpus image, the top-10 pool will be semantically weaker and the stego may look less natural. The 1,000-image corpus was sufficient for the three evaluation classes here; a deployment would need a larger and more diverse collection, or a retrieve-or-generate fallback that uses diffusion only when no corpus image is close enough. The exact layer, for its part, requires a lossless channel: it suits file transfer, not photo sharing that recompresses images.

6.8 Deviations from the Paper

Table 6.6: Documented differences from the paper and their reason.

Paper	**This project**	**Reason**
512 px	256 px	Stable Diffusion + ControlNet exceeds 6 GB at 512 px
EDICT 50 steps, strength 0.5	20 steps, strength 0.2	Run time and memory; a configuration value
SwinJSCC $C = 48$, CelebA-HQ	$C = 64$, high-resolution release	Nearest public checkpoint
MDJCM digital branch	Not reproduced	No public checkpoint; no digital result claimed
Unnamed LLM	Qwen2.5-0.5B-Instruct	Local, deterministic, fits memory
OpenPose on all faces	MediaPipe on 3 of 22	OpenPose finds no body on tight crops
CodeFormer on GPU	On CPU	GPU memory
XuNet details	Public XuNet design, image-grouped patches	Not fully specified; small test set

6.9 Limitations

- Quality comparisons use 2 images per class (24 cases per system); the full 22-per-class run is a configuration change requiring several hours.
- Detectability of the exact layer is measured against classical detectors only; a trained CNN steganalyser on this cover source remains to be evaluated.
- The exact layer is fragile by design: recompression destroys the payload (detected, not silently accepted), so the stego must be sent as a file.
- The 8-digit token is short; key stretching slows brute force, but a longer key is advisable in practice.
- One style evaluation image is a human portrait rather than a cat photograph.

Chapter 7

Conclusion

7.1 Summary

This work reproduced AgentSemSteCom, an agentic, generative semantic steganographic communication scheme, end to end on a single 6 GB laptop GPU, using the components named in the paper and documenting every unavoidable difference. The reproduction is in the paper's regime: facial recovery of 22.6–25.1 dB across 5–20 dB SNR, a correct-minus-wrong token gap of 8.41 dB (paper 6.46 dB), and a XuNet steganalyser at chance.

On top of this baseline, the project built **RACS**, which improves it in two ways:

- **Retrieval instead of generation.** The reference image is found in an indexed collection and picked by the token, instead of being painted by a diffusion model. It is obtained about 82× faster (0.048 s instead of 3.92 s) with no GPU memory, sender and receiver get the identical image for all 66 test images, and quality and security are kept or slightly improved (PSNR 22.62 vs 22.37 dB, security gap 8.94 vs 8.67 dB).
- **Exact recovery.** All 66 secrets are recovered pixel-identical, every wrong key is rejected, the chi-square attack is reduced to chance, and Sample Pair Analysis is weakened from AUC 1.00 to 0.83. The cover image's realism turned out to matter as much as the embedding rule.

A phone prototype shows RACS working on real photographs, with a receive path that needs no AI model and finishes in one to two seconds.

7.2 Learning Outcomes

- Practical understanding of latent diffusion, DDIM, exact inversion with EDICT, ControlNet and IP-Adapter, and of where each loses or preserves information.

- Deep joint source–channel coding and channel simulation at a controlled SNR.
- Multimodal retrieval with CLIP and exact nearest-neighbour search, and why determinism matters more than speed when both ends must agree.
- Applied cryptography for steganography: key stretching, stream encryption, message authentication, and the difference between hiding content and hiding existence.
- Steganalysis: chi-square, Sample Pair Analysis and CNN detectors, and how cover-source statistics affect detectability.
- Research engineering: phase gates, frozen splits, configuration and model hashing, a deviation register, and memory management on constrained hardware.

7.3 Future Work

- **Full-scale evaluation.** Run the comparison on all 22 images per class and report confidence intervals; run EDICT at the paper's 50 steps and strength 0.5.
- **RACS variants.** Vision–language reranking with a fixed deterministic ranking (RACS-Agent), lightweight deterministic editing of the retrieved image (RACS-AgentEdit), and a retrieve-or-generate fallback when no corpus image is close enough (RACS-Hybrid).
- **Larger corpora.** Grow the corpus from 1,000 to 50,000–100,000 images and study the trade-off between retrieval quality and index cost under the determinism contract.
- **Stronger steganalysis.** Evaluate the exact layer against trained CNN steganalysers such as SRNet on this cover source, and adapt embedding costs accordingly.
- **Robust exact layer.** Add error-correcting codes so that the exact payload survives mild recompression, and support longer keys.
- **Digital branch.** Reproduce the digital semantic codec when a checkpoint becomes available.

The most natural next step is RACS-Hybrid (Figure 7.1), which keeps retrieval as the fast default and falls back to diffusion only when the corpus

has nothing close enough. The decision must itself be deterministic and computed from public data, so that the receiver always takes the same branch as the sender.

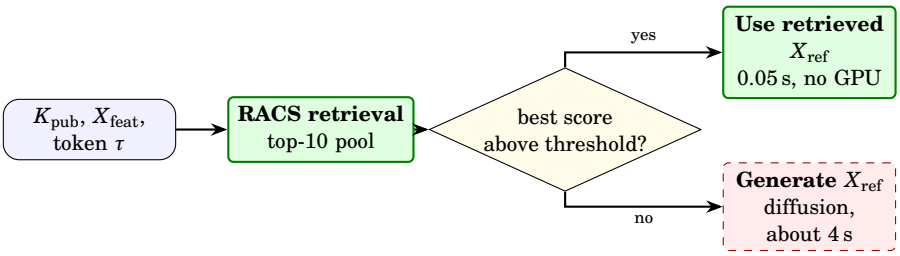


Figure 7.1: Proposed RACS-Hybrid: retrieval by default, generation only as a fallback. The threshold uses public scores only, so both ends make the same choice.

7.4 Conclusion

Diffusion generation is not required to obtain a good steganographic reference image: a deterministic, indexed retrieval stage delivers the same recovery quality and security at a small fraction of the cost and without a GPU, provided the transmitter and receiver follow a strict determinism contract. And approximate generative recovery need not be the end point: a carefully designed authenticated layer can deliver the exact secret while keeping classical detectors near chance. Together, these results make coverless semantic steganography cheaper, exact and more practical for real devices.

Bibliography

[1] Shumeet Baluja. Hiding images in plain sight: Deep steganography. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[2] Eirina Bourtsoulatze, David Burth Kurka, and Deniz Gündüz. Deep joint source-channel coding for wireless image transmission. *IEEE Transactions on Cognitive Communications and Networking*, 5(3):567–579, 2019.

[3] Zhe Cao, Gines Hidalgo, Tomas Simon, Shih-En Wei, and Yaser Sheikh. OpenPose: Realtime multi-person 2D pose estimation using part affinity fields. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(1):172–186, 2021.

[4] Wenhu Chen, Hexiang Hu, Chitwan Saharia, and William W. Cohen. Re-Imagen: Retrieval-augmented text-to-image generator. *arXiv preprint arXiv:2209.14491*, 2022.

[5] Sorina Dumitrescu, Xiaolin Wu, and Zhe Wang. Detection of LSB steganography via sample pair analysis. *IEEE Transactions on Signal Processing*, 51(7):1995–2007, 2003.

[6] Jessica Fridrich. *Steganography in Digital Media: Principles, Algorithms, and Applications*. Cambridge University Press, 2009.

[7] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[8] Jitesh Jain, Jiachen Li, MangTik Chiu, Ali Hassani, Nikita Orlov, and Humphrey Shi. OneFormer: One transformer to rule universal image segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2989–2998, 2023.

[9] Junpeng Jing, Xin Deng, Mai Xu, Jianyi Wang, and Zhenyu Guan. HiNet: Deep image hiding by invertible network. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 4733–4742, 2021.

[10] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.

[11] Hugo Krawczyk, Mihir Bellare, and Ran Canetti. HMAC: Keyed-hashing for message authentication. RFC 2104, Internet Engineering Task Force, 1997.

[12] Junnan Li, Dongxu Li, Caiming Xiong, and Steven Hoi. BLIP: Bootstrapping language-image pre-training for unified vision-language understanding and generation. In *International Conference on Machine Learning (ICML)*, pages 12888–12900, 2022.

[13] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft COCO: Common objects in context. In *European Conference on Computer Vision (ECCV)*, pages 740–755, 2014.

[14] Shao-Ping Lu, Rong Wang, Tao Zhong, and Paul L. Rosin. Large-capacity image steganography based on invertible neural networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10816–10825, 2021.

[15] Rui Meng, Song Gao, Bingxuan Xu, Xiaodong Xu, Jianqiao Chen, Nan Ma, Pei Xiao, Ping Zhang, and Rahim Tafazolli. Secure intellicise wireless network: Agentic AI for coverless semantic steganography communication. *IEEE Transactions on Cognitive Communications and Networking*, 12:8106, 2026.

[16] Kathleen Moriarty, Burt Kaliski, and Andreas Rusch. PKCS #5: Password-based cryptography specification version 2.1. RFC 8018, Internet Engineering Task Force, 2017.

[17] National Institute of Standards and Technology. SHA-3 standard: Permutation-based hash and extendable-output functions. Technical Report FIPS PUB 202, NIST, 2015.

[18] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[19] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*, pages 8748–8763, 2021.

[20] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10684–10695, 2022.

[21] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations (ICLR)*, 2021.

[22] Bram Wallace, Akash Gokul, and Nikhil Naik. EDICT: Exact diffusion inversion via coupled transformations. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 22532–22541, 2023.

[23] Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004.

[24] Andreas Westfeld and Andreas Pfitzmann. Attacks on steganographic systems. In *Information Hiding*, volume 1768 of *Lecture Notes in Computer Science*, pages 61–76, 1999.

[25] Guanshuo Xu, Han-Zhou Wu, and Yun-Qing Shi. Structural design of convolutional neural networks for steganalysis. *IEEE Signal Processing Letters*, 23(5):708–712, 2016.

[26] Ke Yang, Sixian Wang, Jincheng Dai, Xiaoqi Qin, Kai Niu, and Ping Zhang. SwinJSCC: Taming swin transformer for deep joint source-channel coding. *IEEE Transactions on Cognitive Communications and Networking*, 2024.

[27] Yiwei Yang, Zheyuan Liu, Jun Jia, Zhongpai Gao, Yunhao Li, Wei Sun, Xiaohong Liu, and Guangtao Zhai. DiffStega: Towards universal training-free coverless image steganography with diffusion models. In *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*, 2024.

[28] Hu Ye, Jun Zhang, Sibo Liu, Xiao Han, and Wei Yang. IP-Adapter: Text compatible image prompt adapter for text-to-image diffusion models. *arXiv preprint arXiv:2308.06721*, 2023.

[29] Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. Adding conditional control to text-to-image diffusion models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 3836–3847, 2023.

[30] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 586–595, 2018.

[31] Shangchen Zhou, Kelvin C. K. Chan, Chongyi Li, and Chen Change Loy. Towards robust blind face restoration with codebook lookup transformer. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[32] Zhili Zhou, Huiyu Sun, Rohan Harit, Xianyi Chen, and Xingming Sun. Coverless image steganography without embedding. In *International Conference on Cloud Computing and Security*, pages 123–132, 2015.